My Project

Generated by Doxygen 1.17.0

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

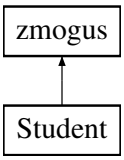
Here is a list of all documented files with brief descriptions:

Chapter 4

Class Documentation

4.1 Student Class Reference

Inheritance diagram for Student:



Public Member Functions

- **Student** (const std::string &v, const std::string &p, const Vector< double > &nd, double egz)
- **Student** (const Student &other)
- **Student** (Student &&other) noexcept
- Student & **operator=** (const Student &other)
- Student & **operator=** (Student &&other) noexcept
- void **skaiciuoti** ()
- double **vid** () const
- double **med** () const
- void printInfo () const override

Public Member Functions inherited from zmogus

- **zmogus** (const std::string &v, const std::string &p)
- const std::string & **vardas** () const
- const std::string & **pavarde** () const

Friends

- std::ostream & **operator**<< (std::ostream &os, const Student &s)
- std::istream & **operator**>> (std::istream &is, Student &s)

Additional Inherited Members

Protected Attributes inherited from zmogus

- std::string **vardas_**
- std::string **pavarde_**

4.1.1 Member Function Documentation

4.1.1.1 printInfo()

```
void Student::printInfo () const  [override], [virtual]
```

Implements zmogus.

The documentation for this class was generated from the following files:

- header.h
- header.cpp

4.2 Vector< T > Class Template Reference

Public Types

- using **iterator** = T*
- using **const_iterator** = const T*

Public Member Functions

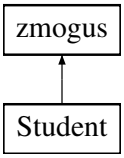
- **Vector** (size_t n)
- **Vector** (std::initializer_list< T > init)
- **Vector** (const Vector &other)
- **Vector** (Vector &&other) noexcept
- Vector & **operator=** (const Vector &other)
- Vector & **operator=** (Vector &&other) noexcept
- T & **operator[ ]** (size_t index)
- const T & **operator[ ]** (size_t index) const
- T & **at** (size_t index)
- const T & **at** (size_t index) const
- T & **back** ()
- const T & **back** () const
- iterator **begin** ()
- const_iterator **begin** () const
- iterator **end** ()
- const_iterator **end** () const
- size_t **size** () const
- size_t **capacity** () const
- bool **empty** () const
- void **reserve** (size_t new_cap)
- void **resize** (size_t new_size)
- void **push_back** (const T &value)
- void **push_back** (T &&value)
- void **pop_back** ()
- void **clear** ()
- void **swap** (Vector &other) noexcept

The documentation for this class was generated from the following file:

- vector.h

4.3 zmogus Class Reference

Inheritance diagram for zmogus:



Public Member Functions

- **zmogus** (const std::string &v, const std::string &p)
- const std::string & **vardas** () const
- const std::string & **pavarde** () const
- virtual void **printInfo** () const =0

Protected Attributes

- std::string **vardas_**
- std::string **pavarde_**

The documentation for this class was generated from the following file:

- zmogus.h

Chapter 5

File Documentation

5.1 header.h

```
00001 #ifndef HEADER_H
00002 #define HEADER_H
00003
00004 #include "zmogus.h"
00005 #include <string>
00006 #include "vector.h"
00007
00008 class Student : public zmogus {
00009 private:
00010     Vector<double> nd_;
00011     double egz_;
00012     double vid_;
00013     double med_;
00014
00015 public:
00016     Student();
00017     Student(const std::string& v, const std::string& p, const Vector<double>& nd, double egz);
00018
00019     //Rule of five
00020     Student(const Student& other);
00021     Student(Student&& other) noexcept;
00022
00023     Student& operator=(const Student& other);
00024     Student& operator=(Student&& other) noexcept;
00025
00026     ~Student();
00027
00028     void skaiciuoti();
00029
00030     double vid() const;
00031     double med() const;
00032
00033     void printInfo() const override;
00034
00035     friend std::ostream& operator«(std::ostream& os, const Student& s);
00036     friend std::istream& operator»(std::istream& is, Student& s);
00037 };
00038
00039 int s_int();
00040 double s_double();
00041 int atsitiktinis();
00042
00043 void spausdinti_lentele(const Vector<Student>& A, int pas);
00044 void generuoti_studentus(int kiekismok, int kiekpaz);
00045
00046 #endif
```

5.2 vector.h

```
00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <algorithm>
```

```
00005 #include <stdexcept>
00006 #include <initializer_list>
00007
00008 template <typename T>
00009 class Vector {
00010 private:
00011     T* data_;
00012     size_t size_;
00013     size_t capacity_;
00014
00015     void reallocate(size_t new_cap) {
00016         T* new_data = new T[new_cap];
00017         for (size_t i = 0; i < size_; ++i)
00018             new_data[i] = std::move(data_[i]);
00019         delete[] data_;
00020         data_ = new_data;
00021         capacity_ = new_cap;
00022     }
00023
00024 public:
00025     using iterator = T*;
00026     using const_iterator = const T*;
00027
00028
00029     Vector() : data_(nullptr), size_(0), capacity_(0) {}
00030
00031
00032     explicit Vector(size_t n) : data_(new T[n]), size_(n), capacity_(n) {
00033         for (size_t i = 0; i < size_; ++i)
00034             data_[i] = T();
00035     }
00036
00037     Vector(std::initializer_list<T> init) : data_(new T[init.size()]), size_(init.size()),
      capacity_(init.size()) {
00038         size_t i = 0;
00039         for (const T& val : init)
00040             data_[i++] = val;
00041     }
00042
00043
00044     Vector(const Vector& other) : data_(new T[other.size_]), size_(other.size_),
      capacity_(other.size_) {
00045         for (size_t i = 0; i < size_; ++i)
00046             data_[i] = other.data_[i];
00047     }
00048
00049
00050     Vector(Vector&& other) noexcept : data_(other.data_), size_(other.size_),
      capacity_(other.capacity_) {
00051         other.data_ = nullptr;
00052         other.size_ = 0;
00053         other.capacity_ = 0;
00054     }
00055
00056
00057     ~Vector() {
00058         delete[] data_;
00059     }
00060
00061     // Copy assignment
00062     Vector& operator=(const Vector& other) {
00063         if (this != &other) {
00064             Vector temp(other);
00065             swap(temp);
00066         }
00067         return *this;
00068     }
00069
00070     // Move assignment
00071     Vector& operator=(Vector&& other) noexcept {
00072         if (this != &other) {
00073             delete[] data_;
00074             data_ = other.data_;
00075             size_ = other.size_;
00076             capacity_ = other.capacity_;
00077             other.data_ = nullptr;
00078             other.size_ = 0;
00079             other.capacity_ = 0;
00080         }
00081         return *this;
00082     }
00083
00084     // Element access
00085     T& operator[](size_t index) { return data_[index]; }
00086     const T& operator[](size_t index) const { return data_[index]; }
00087
00088     T& at(size_t index) {
```

```
00089         if (index >= size_)
00090             throw std::out_of_range("Vector::at");
00091         return data_[index];
00092     }
00093
00094     const T& at(size_t index) const {
00095         if (index >= size_)
00096             throw std::out_of_range("Vector::at");
00097         return data_[index];
00098     }
00099
00100     T& back() {
00101         return data_[size_ - 1];
00102     }
00103
00104     const T& back() const {
00105         return data_[size_ - 1];
00106     }
00107
00108     // Iterators
00109     iterator begin() { return data_; }
00110     const_iterator begin() const { return data_; }
00111     iterator end() { return data_ + size_; }
00112     const_iterator end() const { return data_ + size_; }
00113
00114     // Capacity
00115     size_t size() const { return size_; }
00116     size_t capacity() const { return capacity_; }
00117     bool empty() const { return size_ == 0; }
00118
00119     void reserve(size_t new_cap) {
00120         if (new_cap > capacity_)
00121             reallocate(new_cap);
00122     }
00123
00124     void resize(size_t new_size) {
00125         if (new_size > capacity_)
00126             reallocate(new_size);
00127         if (new_size > size_) {
00128             for (size_t i = size_; i < new_size; ++i)
00129                 data_[i] = T();
00130         }
00131         size_ = new_size;
00132     }
00133
00134     // Modifiers
00135     void push_back(const T& value) {
00136         if (size_ >= capacity_) {
00137             size_t new_cap = (capacity_ == 0) ? 1 : capacity_ * 2;
00138             reallocate(new_cap);
00139         }
00140         data_[size_++] = value;
00141     }
00142
00143     void push_back(T&& value) {
00144         if (size_ >= capacity_) {
00145             size_t new_cap = (capacity_ == 0) ? 1 : capacity_ * 2;
00146             reallocate(new_cap);
00147         }
00148         data_[size_++] = std::move(value);
00149     }
00150
00151     void pop_back() {
00152         if (size_ > 0)
00153             --size_;
00154     }
00155
00156     void clear() {
00157         size_ = 0;
00158     }
00159
00160     void swap(Vector& other) noexcept {
00161         std::swap(data_, other.data_);
00162         std::swap(size_, other.size_);
00163         std::swap(capacity_, other.capacity_);
00164     }
00165 };
00166
00167 #endif
```

5.3 zmogus.h

```
00001 #ifndef PERSON_H
```

```
00002 #define PERSON_H
00003
00004 #include <iostream>
00005 #include <string>
00006
00007 class zmogus {
00008 protected:
00009     std::string vardas_;
00010     std::string pavarde_;
00011
00012 public:
00013     zmogus () = default;
00014     zmogus(const std::string& v, const std::string& p): vardas_(v), pavarde_(p) {}
00015
00016     virtual ~zmogus() = default;
00017
00018     const std::string& vardas() const { return vardas_; }
00019     const std::string& pavarde() const { return pavarde_; }
00020
00021     virtual void printInfo() const = 0;
00022 };
00023
00024 #endif
```

Index